

Contents

guide YaRN: Efficient Context Window Extension of Large Language Models

0. 这篇论文的位置

1. 当时为什么需要它

2. RoPE 最小数学直觉

3. 为什么 PI 不够

4. NTK-aware 的动机

5. NTK-by-parts: YaRN 的骨架

6. YaRN 再加了 attention scaling

7. 张量级图: RoPE 频率到底怎么变

8. 本地代码怎么读

9. 训练设计

10. 实验证据链

11. Dynamic Scaling 是什么

12. 设计理由总结

13. 局限和注意事项

14. 对今天的意义

15. 常见误解

16. 闭卷检查

17. 用 AI agent 学这篇的正确方式

1

1

2

2

3

3

4

5

5

6

7

7

8

8

8

9

9

9

10

guide_YaRN: Efficient Context Window Extension of Large Language Models

原论文: YaRN: Efficient Context Window Extension of Large Language Models

作者: Bowen Peng, Jeffrey Quesnelle, Honglu Fan, Enrico Shippole

年份: 2023

本地原文 PDF: [learning/long-context/paper/01_yarn_context_extension.pdf](#)

本地代码入口:

[learning/long-context/src/rope_yarn.py](#)

[learning/long-context/src/rope_pi.py](#)

[learning/long-context/src/rope_ntk.py](#)

[learning/long-context/src/capstone_yarn_llama32.py](#)

0. 这篇论文的位置

YaRN 解决的是 RoPE 模型的 context window extension 问题: 一个模型预训练时只见过固定最大长度, 比如 LLaMA 的 2k 或 Llama 2 的 4k, 能不能用很少继续训练, 把它扩到 32k、64k、128k, 并尽量保留短上下文能力?

这篇不是提出新的 Transformer 架构, 也不是让注意力复杂度变低。它修改的是 rotary position embedding 的频率和 attention scaling, 让模型在更长位置上看到的旋转相位仍然落在可学习、可适应的范围内。

一句话读法:

YaRN = NTK-by-parts RoPE interpolation + attention temperature scaling
+ 少量长文本继续训练

它的核心主张是:

不要把所有 RoPE 维度一刀切地压缩。
高频维度负责近距离相对顺序，少动。
低频维度负责长距离/绝对位置，必须插值。
中间维度平滑过渡。

论文最值得记住的不是某个神奇常数，而是这个频率分工的解释框架。

1. 当时为什么需要它

2023 年开源 LLM 快速普及，但长上下文仍很贵。完整预训练一个长上下文模型需要大量长文档、大量显存和大量训练步。另一条路是拿已有 RoPE 模型做 context extension。

已有方法主要有几类。

第一类是直接外推。模型预训练只见过 0 到 L 的位置，推理时直接把位置编号用到 L 之外。RoPE 数学上可以给任意位置算角度，但模型参数没有学过这些分布，输出通常会退化。

第二类是 Position Interpolation, 简称 PI。它把新长窗口的位置压缩回旧窗口，例如新窗口长度是 L' ，旧窗口是 L ，就把位置 m 映射成：

$$m' = \frac{mL}{L'}$$

直觉是：不让模型看到超出预训练范围的位置，而是把 32k 的坐标压回 2k 或 4k 范围。PI 很简单，也能配合少量 fine-tuning 生效，但它把所有 RoPE 维度等比例压缩，会损失高频信息。

第三类是 NTK-aware scaling。它不直接压缩 position，而是改 RoPE 的 base，让不同频率维度承担不同程度的缩放压力。这个方向效果更好，但需要经验性地找 base，并且某些维度可能仍出现 out-of-bound 的外推。

YaRN 的贡献是把这些经验整理成更明确的机制：按 RoPE 维度的 wavelength 分段决定是否插值，再加 attention temperature 修正 long-context 下的注意力分布。

2. RoPE 最小数学直觉

RoPE 把每一对 hidden dimension 看成一个二维平面，在这个平面里按位置旋转 query 和 key。

对第 d 个 RoPE 频率，位置 m 的旋转角大致是：

$$\text{angle}(m, d) = m\theta_d$$

其中：

$$\theta_d = b^{-2d/|D|}$$

b 通常是 10000， $|D|$ 是 head dimension。位置越大，旋转角越大。不同 d 有不同频率，所以有些维度转得很快，有些维度转得很慢。

RoPE 的一个重要性质是 query 和 key 的点积主要依赖相对距离 $m - n$ ，这让它适合表达相对位置。但是论文指出，现实模型里 RoPE 不只是相对位置编码。某些低频维度的 wavelength 很长，在预训练窗口内甚至没完成一圈旋转，这些维度可能保留了类似绝对位置的信息。

为了讨论每个维度转得多快，论文定义 wavelength：

$$\lambda_d = \frac{2\pi}{\theta_d}$$

λ_d 表示第 d 个频率转满一圈需要多少 token。高频维度 λ 小，短距离内就能转很多；低频维度 λ 大，很长距离才转一圈。

3. 为什么 PI 不够

PI 的动作是把所有位置统一压缩:

```
old max length: L
new max length: L'
scale factor: s = L' / L
```

```
new position m -> compressed position m / s
```

它的好处是避免直接外推。比如原模型只见过 4k，想扩到 32k，就把 32k 位置压到 4k 范围。

问题是所有 RoPE 维度都被同样压缩。高频维度原本负责邻近 token 的细粒度顺序，压缩以后高频成分被抹平，模型区分局部相对距离的能力下降。论文把这个问题称为 loss of high frequency information。

可以用一个小图理解:

原始 RoPE:

高频维度: 0 1 2 3 4 5 6 7 位置变化很明显
低频维度: 很慢地变化

PI 后:

所有维度的位置都除以 s
高频维度也被放慢
局部顺序信号变钝

所以 PI 是一个稳健的起点，但它太粗糙。论文提到，使用 PI 的一些长上下文 fine-tuning 大约到 $s = 8$ 后输出开始明显退化，即使继续训练也难完全恢复。

4. NTK-aware 的动机

NTK-aware scaling 的想法是: 不要让所有维度承担同样的压缩。高频维度少缩放，低频维度多缩放，把扩展压力分散到多个频率上。

在 RoPE 里，改 base b 会改变所有频率:

$$\theta_d = b^{-2d/|D|}$$

如果把 b 改成更大的 b' ，低频维度变化更明显，高频维度相对保留得更多。论文在 appendix 中给出一种常见选择:

$$b' = b \cdot s^{|D|/(|D|-2)}$$

这个方法比 PI 更注意高频信息，但它有两个问题。

第一，最佳 base 往往要经验搜索。你知道目标扩展倍数 s ，但不一定知道该用哪个 b' 最合适。

第二，它不是纯插值。某些维度可能仍然进入模型没见过的 out-of-bound 区域，所以 fine-tuning 后不一定比 PI 稳。

YaRN 保留了 NTK-aware 的思想，但把“哪些维度该动、哪些不该动”显式写出来。

5. NTK-by-parts: YaRN 的骨架

论文用一个比 wavelength 更直观的量：

$$r(d) = \frac{L}{\lambda_d}$$

$r(d)$ 表示在预训练最大长度 L 内，第 d 个维度大约转了几圈。

如果 $r(d)$ 很大，说明该维度 wavelength 很短，在预训练窗口内已经转了很多圈。它主要表达局部相对位置信息。这样的维度不应该插值，否则会破坏近距离顺序感。

如果 $r(d)$ 很小，说明该维度 wavelength 很长，在预训练窗口内可能还没转完一圈。它更像绝对位置或长距离信号。扩展窗口时如果不插值，新位置会进入分布外，所以应该像 PI 那样压缩。

中间维度介于两者之间，需要平滑过渡。

论文引入两个阈值 α 和 β ，对 LLaMA/Llama 2 推荐 $\alpha = 1$, $\beta = 32$ 。然后定义 ramp function:

$$\gamma(r) = \begin{cases} 0, & r < \alpha \\ 1, & r > \beta \\ \frac{r-\alpha}{\beta-\alpha}, & otherwise \end{cases}$$

再用它混合“插值后的频率”和“原始频率”：

$$h(\theta_d) = (1 - \gamma(r(d)))\frac{\theta_d}{s} + \gamma(r(d))\theta_d$$

读这个公式时要抓住三种情况：

$r < \alpha$:

$\gamma = 0$

$h(\theta) = \theta / s$

低频维度完全插值，避免长位置外推

$r > \beta$:

$\gamma = 1$

$h(\theta) = \theta$

高频维度保持原样，保护局部相对距离

$\alpha \leq r \leq \beta$:

在 θ/s 和 θ 之间线性过渡

这就是 NTK-by-parts 的本质：按频率分段，不是一刀切。

6. YaRN 再加了 attention scaling

论文还观察到，插值后 attention logits 的分布会受影响。它引入一个 temperature t ，把注意力权重改成：

$$\text{softmax} \left(\frac{q_m^T k_n}{t \sqrt{|D|}} \right)$$

然后用一个 implementation trick: 不直接改 attention 代码，而是通过缩放 rotary embedding 等价地缩放 query 和 key。这样可以保持对 Flash Attention 2 这类库的兼容。

论文给 LLaMA/Llama 2 推荐的经验式是：

$$\sqrt{\frac{1}{t}} = 0.1 \ln(s) + 1$$

所以 YaRN 的完整定义是：

YaRN = NTK-by-parts interpolation + attention temperature scaling

注意，本地教学代码里 rope_yarn.py 和 capstone_yarn_llama32.py 对这个缩放量的命名偏教学化。学习论文时请以 Eq. 14 和 Eq. 15 的关系为准：论文关心的是 logits denominator 里的 t ，实现里常见的是把可乘到 RoPE embedding 上的 factor 单独命名。

7. 张量级图：RoPE 频率到底怎么变

假设一个 attention head 的维度是 8。RoPE 会把它分成 4 个二维旋转平面：

head_dim = 8

dimension pairs:

```
pair 0: x[0], x[1]  -> theta_0, high frequency
pair 1: x[2], x[3]  -> theta_1
pair 2: x[4], x[5]  -> theta_2
pair 3: x[6], x[7]  -> theta_3, low frequency
```

原始 RoPE:

```
angles[pos, pair] = pos * theta[pair]
cos = cos(angles)
sin = sin(angles)
```

PI:

```
angles[pos, pair] = (pos / s) * theta[pair]
```

NTK-aware:

```
theta[pair] is rebuilt from a larger base b'
angles[pos, pair] = pos * theta_ntk[pair]
```

YaRN:

```

for each pair:
    lambda_pair = 2*pi / theta[pair]
    r = original_max_pos / lambda_pair
    gamma = ramp(r, alpha, beta)
    theta_yarn = (1 - gamma) * theta[pair] / s + gamma * theta[pair]

```

```
angles[pos, pair] = pos * theta_yarn[pair]
```

这张图很重要。很多人只记 “YaRN 扩上下文”，但真正要进脑袋的是：它是在 `head_dim` 的 RoPE pair 维度上做分频率处理。

8. 本地代码怎么读

`learning/long-context/src/rope_pi.py` 是 PI 的最小实现。核心是：

```

pos = torch.arange(t, dtype=torch.float32) / scale_factor
angles = pos[:, None] * inv_freq[None, :]

```

它直接把 position 除以扩展倍数。

`learning/long-context/src/rope_ntk.py` 是 NTK-aware 的最小实现。核心是：

```

new_base = base * (scale_factor ** (dim / max(dim - 2, 1)))
inv_freq = 1.0 / (new_base ** (torch.arange(0, dim, 2).float() / dim))

```

它通过改 base 重新生成 inverse frequency。

`learning/long-context/src/rope_yarn.py` 是教学版 YaRN。它先生成原始 `inv_freq`，再构造 YaRN 风格的频率混合：

```

pi_freq = inv_freq / factor
mask = _yarn_ramp(low=0.5, high=0.9, dim=half_d)
inv_freq_yarn = mask * inv_freq + (1 - mask) * pi_freq

```

这份教学代码没有完整复现论文中基于 $r(d) = L/\lambda_d$ 、 α 、 β 的边界计算，而是用一个简单 ramp mask 表达 “部分维度保留、部分维度插值” 的思想。真正读论文时，你要把它理解成概念 skeleton，而不是生产级 YaRN 实现。

如果要按论文写一个更直观的伪代码，可以这样：

```

import math
import torch

def yarn_inv_freq_paper(dim, base, factor, original_max_pos,
                        alpha=1.0, beta=32.0):
    d = torch.arange(0, dim, 2).float()
    theta = 1.0 / (base ** (d / dim))
    wavelength = 2 * math.pi / theta
    r = original_max_pos / wavelength

    gamma = torch.clamp((r - alpha) / (beta - alpha), 0.0, 1.0)
    theta_yarn = (1 - gamma) * (theta / factor) + gamma * theta
    return theta_yarn

def yarn_rope_tables(seq_len, dim, factor, original_max_pos):

```

```

inv_freq = yarn_inv_freq_paper(
    dim=dim,
    base=10000.0,
    factor=factor,
    original_max_pos=original_max_pos,
)
pos = torch.arange(seq_len).float()
angles = pos[:, None] * inv_freq[None, :]
mscale = 0.1 * math.log(factor) + 1.0
return angles.cos(), angles.sin(), mscale

```

learning/long-context/src/capstone_yarn_llama32.py 展示了工程注入路线: 改 model.config.max_position_ 设置 rope_scaling, 替换 attention layer 里的 rotary_emb.inv_freq, 再用 LoRA 做少量长上下文适配。它是 capstone skeleton, 不是完整训练脚本。

9. 训练设计

论文的训练设置有三个关键词: 少量训练、长文本 chunk、train short test long。

训练 128k context 的 Llama 2 7B/13B 时, 模型结构不变, 只改 RoPE embedding frequency。对于 $s = 16$, 也就是 4k 到 64k, 作者在 PG19 上用 64k chunks 继续训练 400 steps, global batch size 64, 学习率 2×10^{-5} , 20 steps warmup, AdamW, FSDP 和 Flash Attention 2。

对于 $s = 32$, 也就是 4k 到 128k, 作者不是从头训满 128k 数据, 而是从已经完成的 $s = 16$ checkpoint 继续训练 200 steps。关键点是: 这个 $s = 32$ 模型仍然只用 64k context data 训练, 却能在评测中外推到 128k。

ablation 用 LLaMA 7B, 从 2k 扩到 32k, PG19 切成 32k segments, 训练 400 steps。结果显示 YaRN 比 PI、NTK-aware、NTK-by-parts 收敛更快, loss 更低。

这就是论文标题里 efficient 的含义: 不是 inference 更便宜, 而是 context extension 的继续训练更省。

10. 实验证据链

论文的证据链分几层。

第一层是 long sequence language modeling。作者在 Proof-pile 和 GovReport 上做 sliding window perplexity, 窗口步长 $S = 256$ 。在 Proof-pile 的 10 个至少 128k token 文档上, Llama 2 7B/13B 的 YaRN 模型展示了长窗口 perplexity 能维持稳定。尤其 $s = 32$ 模型只用 64k 数据继续训练, 却在 128k 上仍可用, 说明 YaRN 具备外推和 transfer learning 能力。

表 1 里有一个非常直观的对比: $s = 16$ 模型到 128k 时 perplexity 爆到大于 100, 而 $s = 32$ 模型在 128k 仍保持在约 2.37 到 2.24 的水平。这个数字说明目标扩展倍数和训练路径仍然重要, 不能只靠一个短窗口模型无限外推。

第二层是方法 ablation。作者比较 PI、NTK-aware、NTK-by-parts、YaRN。YaRN 在相同训练步数下通常 perplexity 更低, passkey retrieval 更好。这个实验对应论文的方法叙事: 只改 position interpolation 不够, 只做 NTK-by-parts 还不完整, attention scaling 能进一步稳定。

第三层是 passkey retrieval。这个任务把一个五位数字藏在大量无意义文本中, 看模型能否从长 prompt 里找回。32k LLaMA 7B 模型做 50 次随机位置测试, YaRN 分数高于其他插值方法。对 64k 和 128k 模型, appendix 里进一步做 8k 到 128k 的 passkey 测试, YaRN 7B/13B 在 128k 上仍有很高准确率, 报告中 128k YaRN 的 passkey accuracy 达到约 99.4%。

第四层是短上下文能力保持。论文在 Open LLM Leaderboard 风格的 ARC-Challenge、HellaSwag、MMLU、TruthfulQA 上测试，观察到 YaRN 相比原 Llama 2 baseline 只有很小退化。作者还提到 $s = 16$ 到 $s = 32$ 的平均分数下降约 0.49%，说明长窗扩展没有把短任务能力严重打坏。

第五层是训练效率。Table 4 把不同方法的 A100 GPU-hours 放在一起，YaRN 32k/64k/128k 的训练成本远低于一些长上下文方案。论文摘要中强调 YaRN 比之前方法少用约 10 倍 tokens、少约 2.5 倍 training steps。

11. Dynamic Scaling 是什么

YaRN 主方法之外，论文还讨论 Dynamic Scaling。在线生成时，sequence length 从 1 一步步增加。如果一开始就固定使用最大扩展倍数，比如 128k 的 scale，那么短序列阶段也会被长窗口缩放影响，可能损失短长度表现。

Dynamic Scaling 的想法是每次 forward 根据当前长度更新 scale:

fixed scaling:

```
s = target_context / original_context
```

dynamic scaling:

```
s = max(1, current_length / original_context)
```

这样在当前长度还没超过原窗口时，不需要强行使用长窗口缩放；超过后再逐步增大 scale。论文也提醒，配合 KV cache 时要小心：如果每一步 scale 变化，已经缓存的 key/value 在应用 RoPE 前后的状态会影响正确性。正确实现应注意缓存位置和旋转的顺序。

学习时要分清：

YaRN：

一种 RoPE extension 方法

Dynamic Scaling：

推理时根据当前长度动态选择 scale 的策略

12. 设计理由总结

YaRN 的每个部件都有明确动机。

PI 的动机：避免直接使用模型没见过的长位置。

PI 的问题：所有频率一起压缩，损失高频局部信息。

NTK-aware 的动机：高频少动，低频多动，保留高频。

NTK-aware 的问题：base 选择经验化，并可能带来 out-of-bound 外推。

NTK-by-parts 的动机：用 wavelength 和 $r(d)$ 明确判断哪些维度该插值，哪些维度该保留。

attention scaling 的动机：RoPE 插值会改变 attention logits 的有效分布，需要用温度校正。

少量 fine-tuning 的动机：位置编码修改后，模型需要适应新分布，但不需要重新预训练。

13. 局限和注意事项

第一，YaRN 不是无限上下文。它可以外推，但外推能力仍受训练长度、数据、模型规模和任务类型影响。论文中 $s = 16$ 模型到 128k perplexity 爆掉，就是提醒。

第二, perplexity 不能完全代表长上下文能力。论文在 passkey 部分指出, 有些模型 perplexity 变差后仍能做检索, 说明长上下文评估必须同时看语言建模和检索/任务表现。

第三, YaRN 不降低 attention 的二次复杂度。上下文变长后, attention 计算和 KV cache 仍然更贵。它解决的是位置泛化和少量训练适配, 不是 FlashAttention 或 PagedAttention 那类系统效率问题。

第四, 不同模型族的最佳 α, β 和 scaling 细节可能不同。论文给 LLaMA/Llama 2 推荐值, 但这不是所有架构的公理。

第五, 真实工程实现要仔细处理 RoPE cache、KV cache、FlashAttention 兼容、tokenizer 和训练数据 packing。本仓库代码是教学骨架, 不能直接当生产级长上下文训练方案。

14. 对今天的意义

YaRN 是长上下文模型工程中的经典节点。它把“扩上下文”从粗暴的 position interpolation, 推进到更细的频率分工: 不同 RoPE 维度承担不同语义, 扩展时不能一刀切。

今天很多模型配置里都有 rope_scaling、rope_theta、yarn、longrope、ntk 等关键词。理解 YaRN 后, 你看到这些配置时不会只觉得是玄学超参, 而能问出正确问题:

- 原窗口是多少?
- 目标窗口是多少?
- 是纯推理外推, 还是继续训练?
- 高频局部相对距离有没有被保留?
- 长上下文评测只看 perplexity, 还是也看 retrieval?
- 系统层面能不能承受更长 KV cache?

YaRN 也提醒一个很重要的学习习惯: 长上下文不是一个单点技术, 它连接位置编码、训练数据、优化预算、评测任务和推理系统。

15. 常见误解

误解一: YaRN 改了 attention 结构。

不准确。它主要改 RoPE frequency 和 attention scaling, 保持模型架构不变, 也保持对 FlashAttention 2 这类库的兼容。

误解二: 只要设置更大的 context length 就行。

不行。模型需要合理的位置编码缩放, 并通常需要少量长文本继续训练。

误解三: 长上下文能力只看 passkey。

passkey 检查检索能力, 但不等价于长文推理、摘要、代码理解或多段证据整合。论文同时看 perplexity、passkey 和标准短任务 benchmark。

误解四: PI 和 YaRN 差不多。

PI 是全维度等比例位置压缩。YaRN 是按 RoPE wavelength 分段处理, 再加 attention scaling。

误解五: YaRN 解决了长上下文计算成本。

没有。位置泛化成功后, 长上下文本身仍会带来更高 attention cost 和 KV cache cost。

16. 闭卷检查

读完后你应该能回答:

1. RoPE 中 θ_d 和 λ_d 分别表示什么?
2. 为什么直接外推位置编号会失败?

3. PI 为什么能缓解外推，又为什么会损失高频信息？
4. NTK-aware scaling 通过改什么量来改变 RoPE 频率？
5. $r(d) = L/\lambda_d$ 的直觉是什么？
6. 为什么 $r(d)$ 很大的维度应该少动？
7. 为什么 $r(d)$ 很小的维度应该插值？
8. YaRN 的 ramp function 在做什么？
9. attention temperature scaling 为什么可以通过缩放 RoPE embedding 实现？
10. train short, test long 在论文实验中具体指什么？
11. 为什么 passkey retrieval 和 perplexity 要一起看？
12. YaRN 为什么不能替代 FlashAttention 或 PagedAttention？

17. 用 AI agent 学这篇的正确方式

不要让 agent 只总结 “YaRN 是高效长上下文扩展”。这句话太浅，进不了脑袋。

第一轮让 agent 手算 RoPE:

请用 `head_dim=8`、`base=10000`、`original_max_pos=4096`、`factor=8` 的例子，列出 4 个 RoPE pair 的 `theta`、`wavelength`、`r=L/lambda`，并说明每个 pair 在 YaRN 中更接近 `theta/s` 还是 `theta`。

第二轮让 agent 对照代码:

请对照 `learning/long-context/src/rope_pi.py`、`rope_ntk.py`、`rope_yarn.py`，逐行说明 PI、NTK-aware、YaRN skeleton 的差异。

重点解释 position scaling、base scaling、frequency mixing 的区别。

第三轮让 agent 出错题:

请给我 8 个判断题，专门考察我是否混淆了

PI、NTK-aware、NTK-by-parts、YaRN、Dynamic Scaling。

我答完后请指出错误概念，并让我重新画频率维度图。

第四轮自己跑本地测试:

```
pytest learning/long-context/src/tests/test_rope_extrapolation.py
```

最后要能闭卷说出这句话:

YaRN 不是简单把位置除以 `scale`，而是根据 RoPE 维度的 `wavelength` 决定低频插值、高频保留、中频过渡，并用 attention scaling 稳定长窗分布。